

UConn · School of Business

Lab 2 — Explaining a Model

SHAP · part lecture, part GitHub

OPIM 5512 · Module 2 · Explainable AI

A good score isn't the finish line — tonight the model explains itself.

The deal

- You already have a model that predicts New England demand well ($R^2 \approx 0.90$). Tonight you make it say **WHY**.
- One partner builds the **GLOBAL** view (what drives the model overall), one builds the **LOCAL** view (why one hour).
- Same GitHub loop as Lab 1: branch → commit → push → pull request → review → merge — then merge both views into one report.
- First ~15 minutes are lecture (what SHAP is). Then it's hands-on.

Your only code tonight is ONE SHAP line — type it. You'll approve a partner's explanation of the same model. Approving what you didn't read is how review dies.

Why it's fair: split a prize

Before the model: you + a friend win a **\$10,000** Kaggle prize. A time machine lets each of you redo it **alone** — you place 2nd (\$7,500), your friend 3rd (\$5,000), nobody → \$0.

team	{ }	{You}	{Friend}	{You, Friend}
prize	\$0	\$7,500	\$5,000	\$10,000

2 players → **2! = 2 orderings** → **each contribution counts ½** (a plain average):

You: +\$7,500 and +\$5,000 → **\$6,250**. Friend: +\$5,000 and +\$2,500 → **\$3,750**. Sum = **\$10,000** ✓

Your fair share = your average contribution across the ways the team could form. This is the Shapley value — a fair split from game theory (Lloyd Shapley, 1953).

Add a player → the weights are FIXED

team	{ }	P1	P2	P3	P1,P2	P1,P3	P2,P3	all
prize	\$0	\$5,000	\$5,000	\$0	\$7,500	\$7,500	\$5,000	\$10,000

3 players → 3! = 6 orderings → weights $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$. Fair shares: P1 \$5,000 · P2 \$3,750 · P3 \$1,250 → **\$10,000** ✓

2 players → 2! = 2 → $\frac{1}{2}, \frac{1}{2}$

3 players → 3! = 6 → $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$

The weights are just 1/n! counting — 3 things → 6 ways to form the team → $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$ for EVERY problem, every dataset. Only the values (v(S)) change. That's why the weights on your worksheet never move.

So what IS a SHAP value?

1. **One number per feature, per prediction** — how far that feature pushed *this* prediction away from the **base value** (the average prediction). Same units as the target: MW.
2. **Signed.** + pushes the prediction up; – pulls it down.
3. **Adds up exactly.** base value + all the SHAP values = the prediction.
4. **Fair.** It's the feature's **average bump** across every team of the other features it could join — the Kaggle-prize split. Features not on the team are filled in from **background rows**.

```
sv = explainer(X_test)    # 149 rows x 6 features

sv.values[i, j]          # feature j's push, hour i (MW)
sv.base_values[i]        # the average prediction

# the receipt always balances:
sv.base_values[i] + sv.values[i].sum()
    == model.predict(X_test)[i]    # True
```

A SHAP value = one feature's fair share of one prediction. Same shape as X: one number per row, per feature.

What SHAP is: a receipt, not a ranking

Permutation importance and partial dependence describe the **model**. SHAP explains **one prediction** — an itemized, **signed** receipt whose line items **add up exactly** to the answer.

base value – the average prediction	14,952 MW
hour_of_day = 17 (5 PM)	+3,000
dewpoint_f = 71 (humid)	+2,300
temp_f = 85 (hot)	+2,200
weekend = 0 (weekday)	+900
wind_kt = 12 (some cooling)	-270
this hour's prediction	23,082 MW

Every prediction gets its own receipt. The line items sum to the answer — the guarantee PI and PDP can't give you.

Three tools, three boss-questions — use all three

PERMUTATION IMPORTANCE

“What matters?”

```
# shuffle a column,  
# watch R2 drop  
permutation_importance(  
    model, X_test, y_test)
```

✓ fast, model-agnostic ranking

✗ global only; correlated
features can fool it

PARTIAL DEPENDENCE

“How does it behave?”

```
# sweep one feature,  
# average the response  
PartialDependenceDisplay  
    .from_estimator(model, X,  
        ["hour_of_day"])
```

✓ shows the SHAPE (up/down/curvy)

✗ an average — hides
who's different

SHAP

“Why THIS one?”

```
sv = shap.TreeExplainer(model)(X)  
shap.plots.waterfall(sv[i]) # 1 row  
shap.plots.beeswarm(sv)    # all
```

✓ per-row + signed + adds up;
rolls up to global for free

✗ costs more compute

PI ranks. PDP shapes. SHAP itemizes. Only SHAP explains a SINGLE prediction — and gives you the global view on the way.

SHAP: give every feature a number — in MW

The model is that **same game**: the players are the **features**, the prize is the **prediction**.

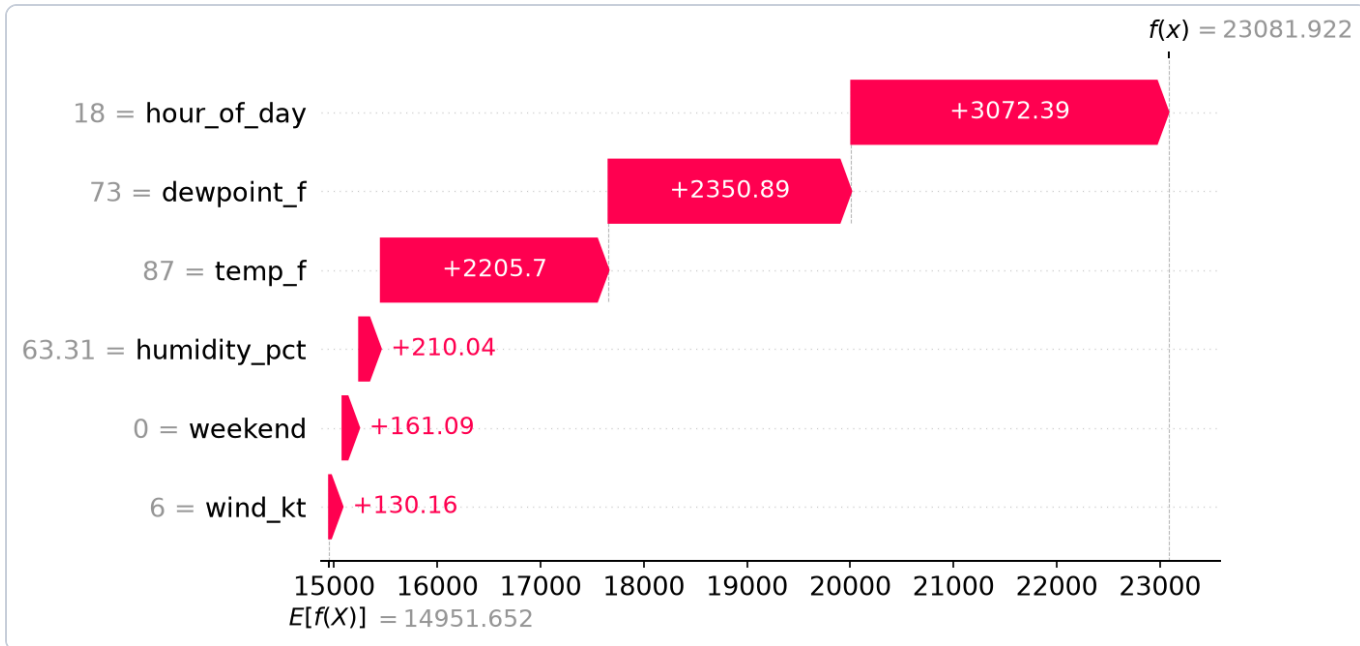
SHAP splits ONE prediction into one number per feature: how many **MW** it pushed the answer up (+) or down (–) from the average prediction. Add every push and you land **exactly** on the answer — additive and honest.

average prediction + (all the feature pushes) = this hour's prediction

14,952 MW + +8,130 MW = 23,082 MW

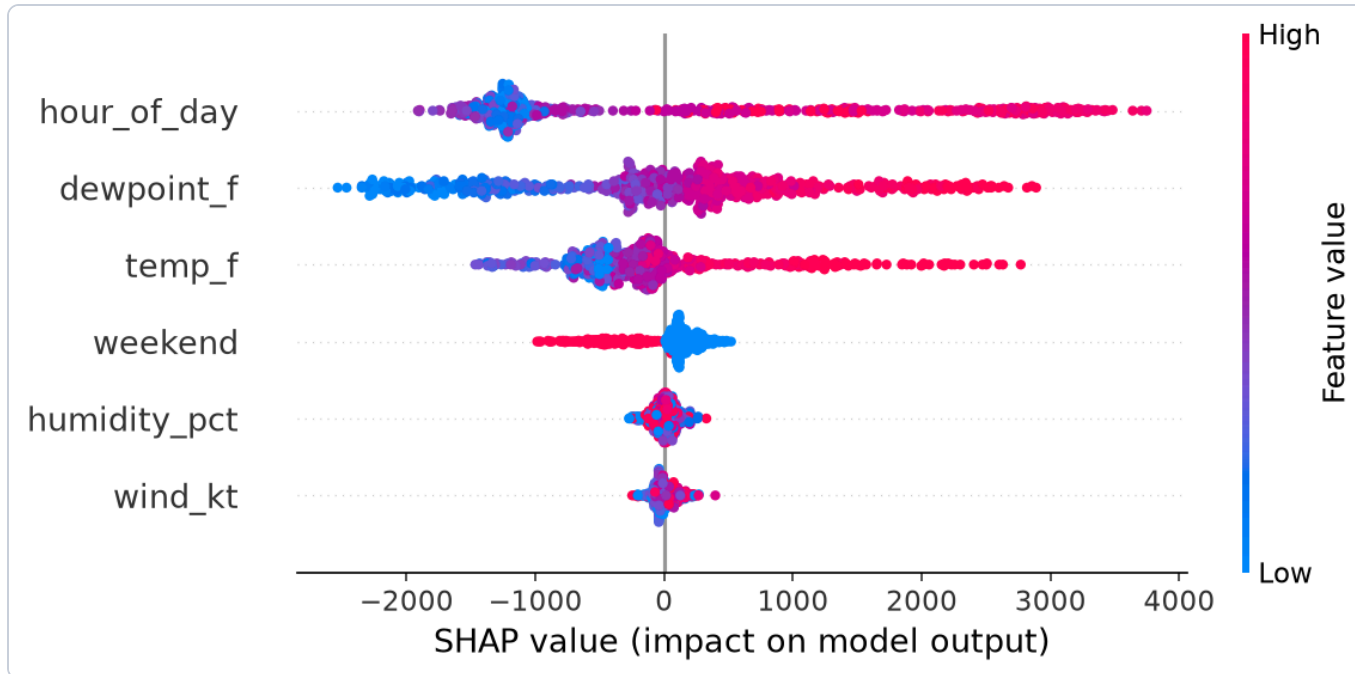
The engine is three lines you DON'T write — the setup cell runs them: `explainer = shap.TreeExplainer(model)` · `shap_values = explainer(X)`

Local: why THIS one prediction (waterfall)



- One hour, one story.
- Start at the average, $E[f(X)]$.
- Each bar is a feature's push in **MW**.
- Land on this hour's prediction, $f(x)$.
- The peak hour is high **because** it's 6 PM, muggy, and hot.
- This is the answer you give a stakeholder.

Global: what drives the model overall (beeswarm)



- Stack every hour's explanation.
- One dot per hour; color = the feature's value (red high, blue low).
- Left/right = push in MW.
- Read top to bottom: **hour_of_day**, **dewpoint**, **temp** do the work.
- High values (red) push demand **up**.
- Global = what it leans on. Local = one story.

Stack check & roles

- **GitHub account** + GitHub Desktop installed and signed in
- **Google Colab** opens in your browser
- **You did Lab 1** — same branch → PR → review → merge loop tonight
- **Pick roles now** — Partner A = GLOBAL (beeswarm), Partner B = LOCAL (waterfall)
- **Have the instructions open** beside Colab — every click is in there, in order

Odd one out pairs with Dave. Online / solo? Two accounts, or pair over Teams.

Make the shared repo — from the template

- 1 • Pair up — Partner A (global) / Partner B (local)
- 2 • Partner A: template repo → Use this template → new PUBLIC repo, owner = you
- 3 • Partner A: Settings → Collaborators → add B → **B ACCEPTS the invite now** (nothing works until B accepts — check spam)
- 4 • Partner A: Settings → Rules → ruleset on main: require a PR + 1 approval
- 5 • Both: clone the repo ONCE in GitHub Desktop
- 6 • Each: New branch (dev-global / dev-local) → Publish branch → **Fetch/Pull**

The template holds the data, both notebooks, images/, README (with the lecture) and REPORT. You build nothing by hand.

Run it, write one SHAP line, ship it

- 7 • Colab → File → Open notebook → GitHub tab → your repo URL → open your notebook
- 8 • Runtime → Run all. Setup installs SHAP, fits the model (note R^2), builds `shap_values` + one free plot
- 9 • In the TODO cell: A → beeswarm → `shap_global.png` ; B → waterfall for the peak hour → `shap_local.png`
- 10 • Look at your plot. Run the download cell → two PNGs land in Downloads
- 11 • File → Save the notebook → your repo · your dev- branch · same path (plain Save, not Ctrl+S)
- 12 • Desktop → Show in Explorer → drag the PNGs into images/ → Commit → Push

One line: `shap.plots.beeswarm(shap_values) / shap.plots.waterfall(shap_values[i])`

The two-way loop

- 13** • Compare & pull request → Create → Reviewers → your partner (*not listed? the invite wasn't accepted — Act 1, step 3*)
- 14** • Open your PARTNER's PR → Files changed → read their SHAP cell, look at the PNG → Approve
- 15** • Merge both PRs → Delete branch
- 16** • Both: Desktop → main → Fetch → Pull — four PNGs and both notebooks are now on your laptop

You can't approve your own PR — that's the gate. Approve only the explanation you actually read. No loop in the network graph? Someone committed to main.

Do global and local agree?

- 17 • One screen: REPORT.md → Edit → one sentence per → line (every number gets a unit)
- 18 • Commit to a new branch 'report' → PR → the other partner approves → Merge
- 19 • Ahead? Run the joint notebook → shap_dependence.png → images/ → report section 5
- 20 • Read-out: beeswarm + waterfall on screen — do they tell the SAME story? Check Insights → Network

Optional bonus — skip tonight if short on time. The two plots + the branch → PR → review → merge loop are the win; the report is extra.

Good advice

- **No module named 'shap'?** The setup cell installs it — Runtime → Run all from the top.
- **SHAP plot errors on shap_values?** You skipped the setup cell that builds it. Run all from the top.
- **Reviewer / PR / branch not showing?** 9 times of 10 the invite wasn't accepted (Act 1). Otherwise GitHub is lagging — wait ~30s and refresh.
- **main rejecting your push is the protection working** — switch to your dev- branch.
- **Empty folder after a new branch?** Fetch origin → Pull in GitHub Desktop.

Online / solo students

- **Pair over Teams if you can** — each on your own account, one owns the repo and adds the other.
- **Otherwise, two accounts** (needs a second email) — play both A and B, including approving as the other account.
- **Simplest solo** — one account with Required approvals = 0: branch → PR → merge yourself.
- **Either way** — join the live session on Teams; Friday office hours are highly recommended.

Definition of done

- ☐ Both partners are collaborators; branch protection on main
- ☐ images/ has FOUR PNGs with the exact filenames (two per partner)
- ☐ Both notebooks saved back with one SHAP line each (beeswarm / waterfall)
- ☐ ≥ 2 merged pull requests (one each), branches deleted, both authoring AND reviewing
- ☐ A network graph showing the loop going both ways
- ☐ (*optional bonus*) REPORT.md — one real sentence under each plot, plus one 'what SHAP can't tell us'